Belief Revision Assignment Report Draft

1. Introduction

When an artificial intelligence receives conflicting information, how should it update its beliefs? If we keep adding information that contradicts itself, the system becomes useless in practice. This is the basic motivation for belief revision.

The AGM framework gives a theory of how beliefs should change rationally. In this setting there are three central operations:

- expansion, which adds a new belief to the belief base
- contraction, which removes a belief while retaining as much knowledge as possible
- revision, which introduces a new belief while keeping the resulting belief base consistent

These operations are constrained by rationality postulates such as Vacuity, Success, Consistency, and Extensionality. In this project, those postulates were not treated as something to check only at the end. They were used to predict how the system should behave in concrete cases, and those predictions became tests.

Our implementation is a belief-base revision engine for propositional logic in symbolic form. It uses formula parsing, CNF conversion, resolution-based entailment checking, and priority-based partial meet contraction, with revision defined through the Levi identity. The result is a system that stays close to the formal material from the course while still being concrete enough to inspect, test, and explain through worked examples. Through the project we worked on created partial outputs throughout to also understand the process of creating a full belief system. The parser, the CNF and AGM all added different functions to the end product.

2. The Belief Revision Agent

2.1 Belief Base

The belief base is implemented in `belief_base.py` as a list of pairs:

`(formula_tree, priority)`

Each formula is represented as an abstract syntax tree produced by the parser. Atomic propositions are stored as `Atom`, negations as `Not`, and binary connectives as `Operator`. This gives a simple internal structure that can be reused by CNF conversion, resolution, and belief change operations.

Each belief also carries an integer priority (we have examples of when it has not). A larger number means that the formula is more entrenched and should be preserved when contraction has a choice about what to remove. This is the main mechanism used to make contraction non-arbitrary.

The belief base supports three operations:

- `expand(phi)`: add `phi` without checking consistency
- `contract(phi)`: remove enough beliefs so that `phi` is no longer entailed
- `revise(phi)`: first contract by the negation of `phi`, then expand by `phi`

This choice of data structure is simple, but it matches the assignment well. The base stores exactly the explicit beliefs, while entailment handles derived beliefs. This is important because the system should be able to infer formulas such as `Flies` from `Bird` and `Bird -> Flies` even when `Flies` was never added directly.

Just as importantly, this split makes behavior easier to predict and test. We can distinguish between what is explicitly stored and what should only appear as a consequence. That is exactly the kind of distinction the tests rely on.

2.2 Logical Entailment

Entailment is implemented in `resolution.py` using resolution refutation. The algorithm checks whether:

`KB |= q`

by adding the negation of the query to the clauses of the belief base and then trying to derive the empty clause. If the empty clause is derived, the query is entailed. If saturation is reached and no new clauses can be produced, the query is not entailed.

The pipeline is:

1. Parse the formula into a syntax tree.
2. Convert the tree to conjunctive normal form.
3. Flatten the CNF tree into a set of clauses.
4. Add the negation of the query.
5. Repeatedly resolve pairs of clauses.

The implementation removes tautological clauses and treats an empty clause as a contradiction. This is enough to support the examples required by the assignment and the AGM tests.

Here too the tests were important. Instead of only checking whether the code runs, we checked whether the entailment engine behaves as logically expected on both positive and negative cases. That gave us confidence that later failures in contraction or revision were not just hidden bugs in entailment (the tests can be seen throughout the codebase).

2.3 Contraction

Contraction is implemented as partial meet contraction guided by priorities. The goal of `contract(phi)` is not necessarily to remove `phi` as a stored formula, but to remove enough support from the belief base so that `phi` is no longer entailed.

The method works in five steps:

1. If `phi` is a tautology, do nothing.
2. If the belief base does not entail `phi`, do nothing.
3. Compute all maximal subsets of the belief base that do not entail `phi`.
4. Score these subsets using the priorities of the formulas they keep.
5. Intersect the best subsets and keep only the formulas that survive all selected remainders.

This is the core AGM-style contraction step. In the current implementation, the selection function is lexicographic: subsets that preserve the highest-priority formulas are preferred. This models epistemic entrenchment in a direct and operational way.

The best short example is the case where both `Bird` and `Bird -> Flies` support `Flies`, but `Bird` has higher priority. After contracting `Flies`, the implication is removed and `Bird` remains. This shows that the system is not deleting beliefs randomly; it is using the priority ordering exactly as intended.

This was also a good example of prediction-first design. Before treating contraction as "working," we asked what should happen in a concrete case where there are multiple possible supports for a conclusion. The predicted behavior was that the weaker belief should be sacrificed first. The corresponding test then checks exactly that.

2.4 Expansion

Expansion is the simplest belief change operation. `expand(phi)` just inserts the new formula into the belief base together with a priority. No consistency check is performed.

This matters because expansion alone is not safe when the new information contradicts existing beliefs. If we already have `Bird`, `Bird -> Flies`, and then simply expand by `Not Flies`, the belief base becomes inconsistent. Under classical logic, an inconsistent theory entails everything, so the system starts answering `YES` even to absurd queries. That example is included in the appendix because it gives the clearest motivation for why revision is needed.

This example was useful not only conceptually but also methodologically. It gave us a concrete failure mode to predict and test against: naive expansion should explode under contradiction, while AGM revision should

avoid that behavior.

3. Implementation

3.1 Parser and Formula Representation

The parser in `parse.py` accepts propositional formulas in prefix notation such as:

```
IMPLIES Bird Flies
NOT Q
AND A (OR B C)
```

Internally, these are turned into syntax trees. For example:

```
IMPLIES Bird Flies
```

becomes:

```
IMPLIES(BIRD, FLIES)
```

The parser normalizes atoms to uppercase and uses explicit node types for atoms, negation, and binary operators. This makes later transformations straightforward because the CNF converter and resolution engine can work directly over the tree structure.

3.2 CNF Conversion

Resolution requires formulas to be in conjunctive normal form. The conversion is implemented in `cnf.py` in three stages:

1. eliminate `IMPLIES` and `BICONDITIONAL`
2. push negation inward using De Morgan's laws and double-negation elimination
3. distribute `OR` over `AND`

For example:

```
IMPLIES A B
```

is converted into:

```
OR Not(A) B
```

and then used as part of the clause extraction process. This staged conversion keeps the code easy to explain and test, since each step corresponds to a standard logical transformation from the course.

3.3 Resolution Engine

The resolution engine extracts clauses as sets of literals. For example, a clause such as:

```
OR A (NOT B)
```

becomes a clause set equivalent to:

```
{A, ~B}
```

The engine then resolves complementary literals across clause pairs until either the empty clause is produced or no progress is possible. This supports both ordinary entailment and the special cases used by contraction and revision.

Example 1 in the appendix is the most direct illustration: from `Bird` and `Bird -> Flies`, the system derives `Flies` although it was never explicitly stored in the belief base.

3.4 Revision

Revision is implemented in `belief_base.py` using the Levi identity:

```
K * phi = (K ÷ NOT phi) + phi
```

This means that new information is not inserted blindly. If the new formula conflicts with the current belief base, the system first contracts by its negation and only then adds the new formula. As a result, the revised belief base accepts the new information while trying to preserve as much entrenched information as possible.

The clearest example is the Tweety case. The original belief base entails `Flies`. Revising with `Not Flies` removes the weaker support for `Flies`, keeps the more entrenched `Bird`, and adds `Not Flies`. In contrast, naive expansion with `Not Flies` would make the base inconsistent and cause logical explosion.

3.5 Testing Against AGM Postulates

The assignment explicitly asks that the algorithm be tested against AGM postulates. In this project, that requirement shaped the implementation process quite strongly. We treated the postulates as behavioral specifications rather than only as theory to mention afterwards. The tests in `tests/test_belief_base.py` were written in a way that makes the expected behavior visible from the test names and scenarios.

The test suite checks the required properties:

- Success
- Inclusion
- Vacuity
- Consistency
- Extensionality

For contraction, the implementation ensures that contracting a formula makes it no longer entailed, unless the target is a tautology. For revision, the implementation ensures that the revising formula is believed afterwards. Vacuity is verified for both contraction and revision. Extensionality is tested by revising or contracting with logically equivalent formulas and comparing the resulting consequences. Consistency is tested by revising with a consistent formula and checking that the revised base does not entail a contradiction.

The key point is that these tests were not only for regression checking. They helped structure the implementation itself. Each major component was easier to build once its expected behavior had been written down in small examples:

- parser tests predict what valid and invalid input should look like
- CNF tests predict the exact result of each transformation stage
- resolution tests predict both entailment and non-entailment cases
- belief revision tests predict the observable behavior of contraction and revision

This made the code easier to debug because failures could be localized. If a revision test failed, we could ask whether the bug came from entailment, CNF conversion, or the contraction policy, rather than treating the system as a black box.

The compact AGM example in the appendix is a useful summary of these tests because it shows vacuity, success, revision success, and consistency in one place.

4. What We Learned

Belief systems are a complex matter that the human brain breaks down at instant speed. If it rains, it is wet; if the sky is blue, the sun is shining. That ease of human reasoning made it easy to underestimate the complexity of the problem on a computational level. At first it seemed close to a set of chained if-statements. But once the programmer is no longer the all-knowing controller, and the system itself has to adapt to contradictory evidence, it becomes a very different problem.

What helped most was that it was usually easier to predict how the system should behave than to implement it immediately. The expected output became the leading principle for the implementation. We used a test-based approach and wrote many failing tests up front, then adjusted the implementation until they passed. Modus ponens, inconsistency under naive expansion, and each AGM postulate were given explicit tests, so that the pytest output read more like a specification of behavior than a list of opaque function names.

That approach pushed the project into smaller parts: parser, CNF conversion, resolution, and belief base. Near the end those parts could be integrated into a system capable of producing non-contradictory revised beliefs. Even then, several iterations were needed to get the behavior right, and early design choices turned out to have downstream consequences.

One clear example was the parser. Polish notation removed the need for operator-precedence handling and made the syntax tree easier to build, but it also locked us into single-word atoms. A sentence like "Mars has little green men" therefore had to be encoded as one token, `MARSHASLITTLEGREENMEN`. Another example came from CNF conversion, where an early distribution pass failed on formulas such as `OR (AND A B) (AND C D)`. That bug only became visible because of a carefully chosen test case.

The assignment was technically demanding, especially because belief revision was not a topic all group members had worked with in depth before. One useful outcome was that we saw much more clearly why syntax trees are worth the effort. Once the formulas were represented structurally, the later steps became far easier to reason about and debug.

5. Conclusion and Further Work

The framework was implemented and tested as a basic belief revision system. It works as a way to create simple belief systems that can represent beliefs, derive consequences, and update themselves when new information arrives. The tests showed that the system behaves in line with the AGM-inspired behavior we aimed for, and that it can serve as a solid basis for further extensions.

Appendix A. Worked Examples

The examples below are the ones most worth keeping. They show the actual behavior of the implementation much more clearly than a general description alone.

Example 1 – Expansion and entailment (modus ponens)

KB: 'Birds fly' and 'Tweety is a Bird'. Resolution should derive 'Flies' even though it was never asserted.

```
--- Belief base ---
  1. IMPLIES(BIRD, FLIES)    [priority: 2]
  2. BIRD    [priority: 1]

--- Queries ---
  KB |= Flies                                    ->  YES
  KB |= Bird                                     ->  YES
  KB |= NOT Bird                                 ->  no
```

This is the cleanest entailment example. It shows that the system does more than store formulas: it derives consequences.

Example 2 – Contraction guided by priority

Both 'Bird' and 'Bird -> Flies' support 'Flies'. To stop entailing 'Flies' we have to drop one of them. The lower-priority formula goes.

```
--- Before contraction ---
  1. IMPLIES(BIRD, FLIES)    [priority: 1]
  2. BIRD    [priority: 5]
  KB |= Flies                                    ->  YES

>>> kb.contract(Flies)

--- After contracting Flies ---
  1. BIRD    [priority: 5]
  KB |= Flies                                    ->  no
  KB |= Bird                                     ->  YES
```

This is the key contraction example because it shows the actual priority policy in action.

Example 3 – AGM revision: Tweety turns out to be a penguin

Old KB derives 'Flies'. New evidence says Tweety does NOT fly. Revise means contract support for 'Flies' and then add 'Not Flies'.

```
--- Before revision ---
  1. IMPLIES(BIRD, FLIES)    [priority: 1]
  2. BIRD    [priority: 5]
  KB |= Flies                                    ->  YES

>>> kb.revise(NOT Flies)

--- After revision ---
  1. BIRD    [priority: 5]
  2. Not(FLIES)    [priority: 6]
  KB |= Flies                                    ->  no
  KB |= NOT Flies                                ->  YES
  KB |= Bird                                     ->  YES
```

This is the main revision example. It shows the Levi identity in a way that is easy to explain in the report.

Example 4 – Naive (non-AGM) expansion: ex falso quodlibet

Same starting KB as Example 3, but instead of revising we just expand with 'Not Flies'. The KB is now logically inconsistent because it contains Bird, Bird -> Flies, and Not Flies all at once. In classical logic that means it entails every formula whatsoever.

```
--- Inconsistent belief base ---
  1. IMPLIES(BIRD, FLIES)    [priority: 1]
  2. BIRD    [priority: 5]
  3. Not(FLIES)    [priority: 5]

--- Watch the KB entail nonsense ---
  KB |= Flies                                        ->  YES
  KB |= NOT Flies                                    ->  YES
  KB |= MarsHasLittleGreenMen                        ->  YES
  KB |= NOT MarsHasLittleGreenMen                    ->  YES
  KB |= AtlantisExists                               ->  YES
  KB |= AND MarsHasLittleGreenMen NOT MarsHasLittleGreenMen  ->  YES
```

Every query above returns YES. The KB has lost all ability to distinguish truth from falsehood. This is exactly what AGM revision is designed to prevent.

Example 5 – AGM revision survives even absurd input

'MarsHasLittleGreenMen' is silly, but it does not contradict the KB, so AGM revision just adds it. The KB stays consistent. Then we revise with its negation; this conflicts with what we just added, so contraction drops the absurd belief before the negation is added. The bird beliefs are completely untouched throughout.

```
--- Initial KB ---
  1. IMPLIES(BIRD, FLIES)    [priority: 1]
  2. BIRD    [priority: 5]

>>> kb.revise(MarsHasLittleGreenMen)

--- After revising in the absurdity ---
  1. IMPLIES(BIRD, FLIES)    [priority: 1]
  2. BIRD    [priority: 5]
  3. MARSHASLITTLEGREENMEN    [priority: 6]
  KB |= MarsHasLittleGreenMen                        ->  YES
  KB |= Flies                                        ->  YES
  KB |= AtlantisExists                               ->  no

>>> kb.revise(NOT MarsHasLittleGreenMen)

--- After revising it back out ---
  1. IMPLIES(BIRD, FLIES)    [priority: 1]
  2. BIRD    [priority: 5]
  3. Not(MARSHASLITTLEGREENMEN)    [priority: 7]
  KB |= MarsHasLittleGreenMen                        ->  no
  KB |= NOT MarsHasLittleGreenMen                    ->  YES
  KB |= Flies                                        ->  YES
```

This one is silly on purpose, and it is useful. It shows that revision reacts to contradiction, not to whether the content sounds reasonable.

Example 6 – AGM postulates on a minimal KB

```
--- Initial KB ---
  1. P    [priority: 1]
  2. Q    [priority: 1]

Vacuity: contracting something the KB does not entail (R) is a no-op.
  1. P    [priority: 1]
  2. Q    [priority: 1]

Success: contracting P removes it from the KB.
  1. Q    [priority: 1]
  KB |= P                                        ->  no

Success (revision): revising with NOT Q makes NOT Q believed.
  1. Not(Q)    [priority: 3]
  KB |= Q                                        ->  no
  KB |= NOT Q                                    ->  YES

Consistency: the revised KB is still consistent.
  KB |= AND R NOT R                              ->  no
```

This is the shortest direct link to the AGM postulates named in the assignment.